

BugLens SDK – Project Documentation

Introduction

BugLens SDK is an Android Software Development Kit (SDK) designed to help developers collect bug reports and crash reports directly from their mobile applications.

The SDK provides an end-to-end solution that allows users to report issues, automatically captures application crashes, collects diagnostic information, uploads reports to a backend server, and presents the collected information through a web-based developer portal.

The project was developed as part of an Android SDK course and demonstrates the complete lifecycle of modern bug reporting systems.

Project Objectives

The main objectives of the project are:

- Allow users to report bugs from inside an Android application.
 - Automatically capture application crashes.
 - Collect useful debugging information.
 - Capture and upload screenshots.
 - Store reports in a centralized backend.
 - Provide developers with a dashboard for monitoring issues.
 - Analyze reports through visual analytics.
-

System Architecture

The system consists of three major components:

1. Android SDK

Responsible for:

- Bug reporting
- Crash reporting
- Screenshot capture
- Device information collection
- Metadata collection
- Communication with the backend

2. Backend Server

Responsible for:

- Receiving reports
- Storing reports
- Managing screenshots
- Providing data to the developer portal

3. Developer Portal

Responsible for:

- Displaying reports
- Visualizing analytics
- Viewing screenshots
- Viewing crash information
- Managing report status

System architecture:

Android Application

↓

BugLens SDK

↓

Retrofit REST API

↓

FastAPI Backend

↓

SQLite Database

↓

React Developer Portal

SDK Features

Manual Bug Reporting

Developers can open the BugLens reporting dialog from any screen inside the application.

Users can submit:

- Bug title
- Description
- Severity level

The report is automatically enriched with additional diagnostic information.

Screenshot Capture

When a report is submitted, BugLens captures a screenshot of the current screen.

The screenshot is uploaded to the backend and can later be viewed from the developer portal.

Device Information Collection

BugLens automatically collects:

- Device model
- Manufacturer
- Android version
- Application version

This information helps developers reproduce and investigate issues.

Metadata Collection

Applications can attach custom metadata to reports.

Example:

- Screen name
- Feature name
- User flow information

Metadata allows developers to identify where issues occur most frequently.

Severity Levels

Each report can be classified using one of the following severity levels:

- Low
- Medium
- High
- Critical

Severity information is displayed throughout the dashboard and analytics system.

Shake To Report

BugLens allows users to open the reporting dialog by physically shaking the device.

The implementation uses the Android accelerometer sensor to detect shake events.

This feature provides a quick and user-friendly reporting experience.

Automatic Crash Reporting

BugLens automatically captures uncaught application exceptions.

Collected information includes:

- Exception type
- Error message
- Full stack trace
- Device information
- Application information
- Metadata

Crash reports are uploaded automatically without requiring user interaction.

Backend Design

The backend is implemented using FastAPI and SQLAlchemy.

Its responsibilities include:

- Receiving bug reports
- Receiving crash reports
- Uploading screenshots
- Storing report information
- Providing report data to the portal

The backend exposes REST endpoints that allow communication between the Android SDK and the developer portal.

Database Design

The project currently uses SQLite as its database solution.

Each report stores:

- Report ID
 - API Key
 - User ID
 - Title
 - Description
 - Severity
 - Report Type
 - Stack Trace
 - Device Information
 - Application Information
 - Screenshot Path
 - Metadata
 - Status
 - Creation Time
-

Developer Portal

The developer portal was developed using React and TypeScript.

The portal provides:

- Report management
 - Screenshot viewing
 - Crash investigation
 - Analytics dashboards
-

Analytics

The portal includes several analytics sections:

Report Analytics

- Total reports
- Open reports
- Resolved reports
- Critical reports

Severity Analytics

- Low reports
- Medium reports
- High reports
- Critical reports

Device Analytics

- Reports by device model
- Reports by manufacturer

Android Analytics

- Reports by Android version

Application Analytics

- Reports by application version

Metadata Analytics

- Reports by screen
- Reports by feature

Crash Analytics

- Bug versus crash distribution
 - Crash statistics
-

Challenges Encountered

Backend Architecture

The initial architecture relied on Firebase services. During development the architecture was redesigned to use a dedicated FastAPI backend and SQL database in order to create a more realistic SDK ecosystem.

Screenshot Uploads

Implementing screenshot uploads required coordinating Android file storage, multipart uploads, backend storage, and portal visualization.

Crash Reporting

Automatic crash reporting required implementing a custom uncaught exception handler capable of collecting crash information before application termination.

Analytics Design

Designing a flexible metadata system required balancing generic key-value storage with meaningful visual analytics.

Future Improvements

Potential future enhancements include:

- PostgreSQL migration
 - Offline report queueing
 - Crash grouping and deduplication
 - Email notifications
 - User session recording
 - Network monitoring
 - Advanced filtering capabilities
-

Conclusion

BugLens SDK successfully demonstrates the implementation of a complete bug reporting and crash reporting platform.

The project combines an Android SDK, a backend server, and a developer portal into a single ecosystem that enables developers to collect, manage, and analyze application issues efficiently.

The final system provides a practical example of how modern bug reporting solutions are designed and implemented in real-world software products.